

Geometric Limits of Knowledge Distillation:

A Theory, Experiments, and Implementation Plan

Working Research Document

July 31, 2026

Abstract

This document takes a fresh look at our ongoing research on the geometric limits of knowledge distillation, continuing the project from a new point of view rather than restarting it. The central idea is that knowledge distillation (KD) exhibits geometric loss floors driven by superposition and width-limited capacity: in regimes where teacher models store many sparse features in strong superposition, the minimum loss achievable by a width-limited student has a non-zero floor determined largely by geometric capacity constraints rather than optimization or data alone. The present phase of the research (i) re-states and refines the geometric minimum-width perspective on KD loss floors; (ii) designs cleaner, more robust experiments and code pipelines spanning toy superposition models, language-model KD, and vision/GNN KD; (iii) situates our claims within current KD and mechanistic interpretability literature; and (iv) commits to either strengthening the evidence for geometric KD limits or discovering where the theory breaks and what alternative explanations fit better. Time is not the constraint; rigor, breadth, and clarity are. We accept that some claims may be weakened or replaced if the data demands it.

Contents

1	Goal and Philosophy	3
2	Theory: What We Think Might Be True	3
2.1	Theoretical Basis	3
2.2	Core Hypothesis	3
2.3	The Minimum-Width Argument (Refined)	4
2.4	Relation to Other Geometric KD Work	4
2.5	Mechanistic Interpretability Context	5
3	Experiment Matrix	5
4	Detailed Experiment Designs and Code Directions	6
4.1	Tier 1 — Revisit and Audit the Core Experiments	6
4.1.1	T1: Toy Superposition Capacity Test	6
4.1.2	Pythia-Style Language Model KD (Audit)	7
4.2	Tier 2 — New Architectures and KD Objectives	8
4.2.1	T2: LM KD (Classification) with Geometry-Aware Feature KD	8
4.2.2	T3: LM KD (Instruction Following)	9
4.2.3	T4: Vision/GNN KD with Geometry-Aware Losses	10
4.3	Tier 3 — Comparing Geometric, Interaction, and Alignment Views	10
4.3.1	Interaction Sparsification vs. Feature Superposition	10
4.3.2	Geometry-Aware Alignment vs. Capacity Limits	11

5	How the Experiments Justify the Theory	11
6	Replication Tasks: Anchoring in Other Papers	11
7	Justification: Why This Project Makes Sense Now	12
7.1	Field Needs and Open Problems	12
7.2	Why Width and Superposition?	12
8	Process and Workflow	12
8.1	Code and Reproducibility	12
8.2	Iterative Development	13
8.3	Iteration and Documentation	13
9	Immediate Next Steps	13

1 Goal and Philosophy

The goal of this phase of the research is to *theoretically and experimentally* test the idea that knowledge distillation has geometric loss floors driven by superposition and width-limited capacity, and to situate this idea alongside other geometric and mechanistic KD work. We are not starting over: we are continuing the ongoing investigation from another point of view, with sharper framing and cleaner infrastructure. Concretely, we will:

- Re-state and refine the geometric minimum-width perspective on KD loss floors.
- Design cleaner, more robust experiments and code pipelines.
- Situate our claims within current knowledge distillation and mechanistic interpretability literature.
- Either strengthen the evidence for geometric KD limits, or find new phenomena and counterexamples.

Our commitment is to one of two outcomes: we either strengthen and refine the “minimum-width” perspective, or we discover where it breaks and identify which alternative explanations fit better. Time is not a constraint; rigor and breadth are. We accept that some claims might be weakened or replaced if the data demands it.

2 Theory: What We Think Might Be True

2.1 Theoretical Basis

Our theory rests on three pillars from the recent literature:

1. **Superposition in neural networks.** Networks store more features than dimensions by overlapping feature directions, with phase transitions as sparsity and importance vary [1]. Under strong superposition, loss naturally scales inversely with model width due to geometric overlaps between feature vectors [2].
2. **Compressed sensing and sparse capacity.** Sparse signals can be recovered from fewer dimensions than features, with capacity depending on sparsity and coherence; this motivates a capacity function $g(\alpha)$ relating supportable feature count to width [3].
3. **Linear representation hypothesis and feature geometry.** Language models form approximately linear feature directions, with inner-product structure encoding concepts; geometry-aware distillation aims to preserve this structure [4, 5].

KD surveys emphasize that student capacity and architecture are a central bottleneck, but provide mostly empirical recipes rather than hard limits; this leaves room for geometric theory [8, 9, 10].

2.2 Core Hypothesis

Hypothesis 1 (H1: Geometric KD loss floors). *In regimes where teacher models store many sparse features in strong superposition, the minimum loss achievable by a width-limited student during KD has a non-zero floor that is largely determined by geometric capacity constraints rather than optimization or data alone [1, 2, 16].*

2.3 The Minimum-Width Argument (Refined)

We retain a version of the minimum-width argument, but treat it as a *structured model* rather than a universal theorem.

Model 1 (M1: Capacity model). *Consider a teacher with F latent features encoded in activations, with sparsity α and importance weights I_i [1, 2]. Under compressed sensing-style assumptions, a layer of width d can support at most $d \cdot g(\alpha)$ features, where $g(\alpha)$ is a capacity factor derived from sparsity and superposition geometry [3, 1]. A student with width d_S is therefore effectively limited to*

$$F_S = \lfloor d_S g(\alpha) \rfloor$$

features; features beyond this must be either merged or dropped.

From Model M1 we obtain two central predictions:

- **Critical width.** The width

$$d_S^* = \frac{F}{g(\alpha)}$$

marks a phase transition: below this width, some teacher features cannot be represented at all [1].

- **Loss floor.** Under importance-ordered allocation, the KD floor is approximated by the aggregate importance of dropped features,

$$L_{\text{geom}}(d_S) \approx \sum_{i > F_S} I_i \mathbb{E}[x_i^2],$$

plus an architecture-dependent baseline B accounting for non-geometric errors (e.g., mismatch in inductive bias) [1, 2].

What changes in this phase. Two things:

1. We clearly mark M1 and H1 as an *approximate capacity model* and a *hypothesis*, respectively—with explicit assumptions about sparsity, measurement, and importance ordering—not universal theorems. The present work will test and refine them.
2. We explicitly compare M1 to alternative models, in particular interaction sparsification in KD [6] and geometry-aware feature alignment [5].

2.4 Relation to Other Geometric KD Work

We connect our geometric perspective to recent KD papers:

- **Geometry-aware representational alignment.** Bhattarai et al. show that many feature KD losses (MSE, CKA [11]) can fail to preserve feature structure even at zero loss, motivating Procrustes and Gram-matrix-based distances as better geometric KD objectives [5].
- **Geometric KD frameworks.** GeomKD-style frameworks and geometric KD for GNNs focus on aligning manifolds or topology between teacher and student—framing KD as topology compression—rather than width-driven capacity limits [7].
- **Unified interaction view of KD.** Wang et al. decompose outputs into interactions and find that KD sparsifies complex interactions (students keep fewer interactions). This is conceptually similar to our claim that students keep a subset of teacher features and interactions under capacity constraints [6].

Our niche. We focus on **capacity-driven geometric limits** as a function of student width and teacher feature statistics, rather than only on geometric alignment losses.

2.5 Mechanistic Interpretability Context

Mechanistic interpretability reviews emphasize (i) features as fundamental units of representation, often discovered via sparse dictionary learning or sparse autoencoders (SAEs) [13, 14], and (ii) superposition as a core hypothesis: networks represent more features than dimensions by overlapping feature vectors [13, 2].

We leverage these ideas by:

- Using SAEs and related tools to measure teacher feature counts, sparsity, and importance distributions [14, 13].
- Treating KD as *feature selection under width constraints*, and asking which features are kept or dropped.

3 Experiment Matrix

We design experiments in three conceptual tiers—(1) revisit and strengthen our existing setups (toy autoencoder + Pythia-style LM), (2) extend to at least one non-LM architecture and alternative KD objectives, and (3) directly compare geometric predictions against interaction-based and alignment-based views of KD—realized concretely as an experiment matrix spanning Toy $\rightarrow$ LM $\rightarrow$ Vision/GNN settings (Table 1).

Tier	Setting	Teacher	Students (widths)	KD objective(s)	Feature measurement
T1	Toy superposition	Small ReLU net	2, 4, 8, 16, 32 dims	Direct reconstruction / KD	Analytic + simple probes
T2	LM KD (classification)	BERT/OPT variant	0.25 $\times$, 0.5 $\times$, 1 $\times$ width	Logit KD; feature KD (MSE, CKA, Procrustes, Gram)	SAEs on multiple layers
T3	LM KD (instruction)	OPT/BERT LLM task	0.25 $\times$, 0.5 $\times$, 1 $\times$ width	Logit + geometry-aware feature KD	SAEs + feature Gram matrices
T4	Vision/GNN KD	ViT/ResNet or GNN	Varied channels/widths	Feature KD vs. geometric KD (topology / Procrustes)	SAEs / sparse coding / alignment metrics

Table 1: Experiment matrix overview. Exact datasets, hyperparameters, and seeds are maintained in a separate `experiments_matrix.md`; this table guides what “makes sense” experimentally.

The rationale per tier:

- **T1:** Test the capacity model in a toy setting where superposition can be observed cleanly [1].
- **T2/T3:** Test KD limits and geometry-aware objectives in realistic LM tasks, following Bhattarai et al. and the KD surveys [5, 8, 9].
- **T4:** Check whether similar geometric limits appear in non-text domains (vision/GNN), borrowing from geometric KD and feature-KD work [7, 8].

4 Detailed Experiment Designs and Code Directions

4.1 Tier 1 — Revisit and Audit the Core Experiments

4.1.1 T1: Toy Superposition Capacity Test

Goal. Reproduce and strengthen the evidence that in a toy model with sparse features and superposition, KD loss floors match the capacity-based prediction; then introduce a KD-style student and compare loss vs. width to capacity predictions.

Design.

- **Synthetic data.** Inputs are sparse combinations of F latent features (e.g., 10–50) with varying importance weights and controlled frequency distributions, following the toy autoencoder / compressed-computation setups of Elhage et al. and Liu et al. [1, 2]. Sparsity is controlled via a Bernoulli mask or structured sparsity.
- **Teacher.** A ReLU MLP that learns to reconstruct or classify based on these latent features.
- **Student.** The same architecture with a smaller hidden width (2, 4, 8, 16, 32 dimensions).
- **KD.** Train the student to match teacher outputs (or intermediate activations).
- **Measurements.** (i) Loss vs. width (train/val); (ii) feature representations and effective feature counts and overlaps via weight-matrix analysis (e.g., singular values) and simple probes. Fit the capacity model M1 and compare predicted vs. observed loss floors.

Refinements over our earlier experiments.

- Systematic variation of sparsity α , feature frequency distributions (power law vs. exponential vs. uniform), and noise [2, 3].
- Clear ablation tables: how sensitive are the predictions to changes in these parameters?

Code skeleton (PyTorch).

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4
5 class ToyTeacher(nn.Module):
6     def __init__(self, in_dim, hidden_dim, out_dim):
7         super().__init__()
8         self.net = nn.Sequential(
9             nn.Linear(in_dim, hidden_dim),
10            nn.ReLU(),
11            nn.Linear(hidden_dim, out_dim)
12        )
13
14    def forward(self, x):
15        return self.net(x)
16
17 class ToyStudent(nn.Module):
18     def __init__(self, in_dim, hidden_dim, out_dim):
19         super().__init__()
20         self.net = nn.Sequential(
21             nn.Linear(in_dim, hidden_dim),
22             nn.ReLU(),
23             nn.Linear(hidden_dim, out_dim)
24        )
```

```

25
26     def forward(self, x):
27         return self.net(x)
28
29 def generate_sparse_features(batch_size, F, sparsity, importance):
30     # x: latent features, sparse
31     mask = (torch.rand(batch_size, F) < sparsity).float()
32     latent = torch.randn(batch_size, F) * importance
33     return latent * mask
34
35 def kd_train_step(student, teacher, optimizer, x):
36     with torch.no_grad():
37         y_teacher = teacher(x)
38     y_student = student(x)
39     loss = nn.MSELoss()(y_student, y_teacher)
40     optimizer.zero_grad()
41     loss.backward()
42     optimizer.step()
43     return loss.item()

```

Listing 1: Toy teacher/student models, sparse-feature data generation, and the KD training step for T1.

Directions.

- Implement data generation following *Toy Models of Superposition* (sparse features, importance weights) [1].
- For each student width, run KD and log the KD loss (train/val) and the estimated feature capacity (e.g., from singular values / weight analysis).
- Compare loss curves vs. width against capacity-based predictions.

4.1.2 Pythia-Style Language Model KD (Audit)

Goal. Revisit the KD experiments on a transformer LM, with cleaner code, more baselines, and explicit robustness checks.

Design.

- **Teacher.** A mid-size open LLM (e.g., Pythia [12], a LLaMA variant, or a similarly accessible model).
- **Students.** Multiple widths, keeping depth and head count fixed for simplicity.
- **KD setups.** (i) Logit KD (classic KD [10]); (ii) feature KD with one or more layers [8, 9].
- **SAE measurement.** Train SAEs on several layers to estimate F , α , and the importance distribution of features [14, 13]. Vary SAE hyperparameters (expansion factor, sparsity penalty) and record how the estimates change.

Justification from the literature. KD surveys stress LLM distillation as a key application and highlight open questions around capacity, emergent abilities, and data regimes [8, 9]. Mechanistic interpretability tools (SAEs, probing) are recognized as useful for measuring internal features [13].

New baselines.

- Optimizer and schedule sweeps (e.g., AdamW vs. Lion, different learning-rate schedules), to show that floors are not purely optimization artifacts.
- Data-regime variants (random subset vs. curated domain vs. synthetic data via dataset-distillation ideas) [9].

Outcome. A quantitative comparison of (i) actual student KD loss vs. predicted $L_{\text{geom}}(d_S) + B$ [1, 2], and (ii) the robustness of the prediction to measurement choices and hyperparameters.

4.2 Tier 2 — New Architectures and KD Objectives

4.2.1 T2: LM KD (Classification) with Geometry-Aware Feature KD

Goal. Test whether the geometric capacity model predicts KD floors, and how geometry-aware losses (Procrustes, Gram) affect those floors.

Design.

- **Teacher.** BERT-base or an OPT variant, as in the geometry-aware KD paper [5].
- **Task.** Text-classification benchmarks (e.g., SST-2, AG News) used in that paper.
- **Students.** Reduced-width models (e.g., 50% and 25% hidden size).
- **KD objectives.** Logit KD (KL divergence); feature KD with Euclidean/MSE, CKA [11], Procrustes distance, and feature Gram-matrix distance [5].

Feature KD code sketch. Assuming access to teacher/student hidden states H_t and H_s of shape $(\text{batch} \times \text{seq_len} \times \text{dim})$:

```
1 def compute_gram(features):
2     # features: [batch, seq_len, dim]
3     # Flatten batch and seq, then compute Gram matrix
4     x = features.reshape(-1, features.size(-1)) # [N, d]
5     gram = x @ x.T # [N, N]
6     return gram
7
8 def procrustes_loss(H_t, H_s):
9     # Simplified Procrustes alignment: align H_s to H_t
10    X = H_t.reshape(-1, H_t.size(-1))
11    Y = H_s.reshape(-1, H_s.size(-1))
12    # Center
13    Xc = X - X.mean(0, keepdim=True)
14    Yc = Y - Y.mean(0, keepdim=True)
15    # Orthogonal Procrustes
16    U, _, Vt = torch.linalg.svd(Yc.T @ Xc)
17    R = U @ Vt
18    Y_aligned = Yc @ R
19    return ((Y_aligned - Xc) ** 2).mean()
20
21 def gram_loss(H_t, H_s):
22    G_t = compute_gram(H_t)
23    G_s = compute_gram(H_s)
24    return ((G_t - G_s) ** 2).mean()
```

Listing 2: Geometry-aware feature KD losses: Gram-matrix and orthogonal-Procrustes distances.

```

1 def kd_step(student, teacher, batch, kd_mode="procrustes", alpha=0.5):
2     input_ids, labels = batch
3     with torch.no_grad():
4         out_t = teacher(input_ids, output_hidden_states=True)
5         out_s = student(input_ids, output_hidden_states=True)
6
7     logits_t = out_t.logits
8     logits_s = out_s.logits
9
10    # Logit KD loss
11    kd_logits = nn.KLDivLoss(reduction="batchmean")(
12        torch.log_softmax(logits_s / 1.0, dim=-1),
13        torch.softmax(logits_t / 1.0, dim=-1)
14    )
15
16    # Feature KD loss
17    H_t = out_t.hidden_states[-1]
18    H_s = out_s.hidden_states[-1]
19
20    if kd_mode == "procrustes":
21        kd_feat = procrustes_loss(H_t, H_s)
22    elif kd_mode == "gram":
23        kd_feat = gram_loss(H_t, H_s)
24    else:
25        kd_feat = ((H_t - H_s) ** 2).mean()
26
27    ce_loss = nn.CrossEntropyLoss()(logits_s, labels)
28    loss = ce_loss + alpha * kd_logits + (1 - alpha) * kd_feat
29
30    loss.backward()
31    return loss.item()

```

Listing 3: KD training step combining cross-entropy, logit KD, and geometry-aware feature KD.

Directions.

- Use the public feature-distillation repository as a baseline for Procrustes/Gram implementations and training scripts [15, 5].
- For each student width and KD mode, measure task accuracy, KD loss, and geometric alignment metrics (Procrustes/Gram).
- Extract teacher feature counts and sparsity with SAEs or sparse coding to estimate F and α , and compare capacity predictions to observed KD performance.

4.2.2 T3: LM KD (Instruction Following)

Goal. Extend T2 to instruction-following tasks (e.g., the OPT tasks in the geometry-aware KD paper [5]) to see whether geometric constraints behave similarly or differently for generative tasks.

The design mirrors T2, with:

- **Teacher.** An OPT-based LLM [5].
- **Task.** Instruction-following benchmarks from that paper.
- **Metrics.** BLEU/ROUGE, instruction-following scores, plus KD losses.

We reuse the KD code skeleton and adapt it to sequence-to-sequence and generative metrics.

4.2.3 T4: Vision/GNN KD with Geometry-Aware Losses

Goal. Test whether geometric capacity limits appear in non-LM domains and how they interact with geometry-aware distillation losses.

Candidate setups.

- **Vision.** Teacher: ResNet-50 or ViT on CIFAR-100 or an ImageNet subset; student: narrower ResNet/ViT with fewer channels; KD: standard feature-based KD (MSE, CKA) vs. geometry-aware KD (Procrustes/Gram in conv/ViT layers) [8, 5].
- **GNN.** Teacher: a GNN model from the geometric KD (topology-compression) line of work [7]; student: narrower hidden dimension and fewer layers; KD: geometry-aware topology KD (from that paper) vs. standard KD.

Design and directions.

- Use existing geometric KD code (GNN or vision) as a starting point for losses and training loops [7, 15].
- Measure teacher feature structure via SAEs or suitable feature-disentangling methods (e.g., sparse coding) over intermediate representations to estimate F and α [13, 14].
- Vary student width and check: (i) does a width-dependent loss floor emerge? (ii) does integrating geometry-aware losses change the geometric floor, or mainly affect the baseline B ?

Justification from the literature. KD is widely used beyond text (vision, multimodal, GNNs) [8]. Geometry-aware distillation methods claim to better preserve feature structures; we can test whether that interacts with capacity limits [5]. Geometric KD for GNNs frames KD as topology compression; we test whether width capacity plays a similar role [7].

4.3 Tier 3 — Comparing Geometric, Interaction, and Alignment Views

4.3.1 Interaction Sparsification vs. Feature Superposition

Goal. Relate our “feature capacity” picture to the “interaction sparsification” mechanism highlighted in recent LLM KD work [6].

Design. For a KD setup (e.g., LLM classification or instruction following):

- Use interaction decomposition (as in Wang et al. [6]) to measure how many interactions the teacher vs. student use.
- Use SAEs to measure how many features are active and how they overlap [14].
- Compare the reduction in the number/complexity of interactions (interaction sparsification) against the reduction in the number of importance-weighted features (geometric capacity).

Questions.

- Are students discarding low-importance features *and* low-impact interactions, consistent with a common capacity mechanism?
- Does adding interaction-aware KD losses (e.g., CIP) change the effective geometric floor [6]?

4.3.2 Geometry-Aware Alignment vs. Capacity Limits

Goal. See whether strong geometry-aware feature-alignment losses can overcome or reshape apparent capacity limits.

Design. In a feature KD setup (e.g., BERT/OPT classification tasks [5]):

- Compare standard feature KD vs. geometry-aware KD (Procrustes, Gram losses).
- Measure student capacity usage (via SAEs) and KD performance at different widths.

Questions.

- Do geometry-aware losses primarily help when width is above critical, or can they significantly reduce loss below d_S^* ?
- If they improve performance below the critical width, does that suggest our capacity model needs revision (e.g., more efficient packing of features)?

5 How the Experiments Justify the Theory

For each tier, we explicitly connect experiments back to theory:

- **T1 (Toy).** Does loss vs. width match capacity-model predictions? Do the phase transitions from *Toy Models of Superposition* show up in KD [1]?
- **T2/T3 (LM).** Estimate teacher feature counts and sparsity and compute the critical width d_S^* [1, 2]. Compare KD floors across widths to predicted geometric floors. Test whether geometry-aware KD (Procrustes/Gram) mostly affects the baseline B or can shift the apparent floor [5].
- **T4 (Vision/GNN).** See whether similar capacity-limited behavior occurs when width/channels are compressed. Check whether topology/geometry losses mitigate or reshape capacity limits [7].

Across all tiers, we also compare the geometric-capacity explanation against alternative views: interaction sparsification (KD as dropping interaction orders) [6] and pure geometric alignment (KD as preserving angles and Gram matrices) [5]. If the experiments show the capacity model systematically over- or under-predicts floors, we adjust the theory accordingly (e.g., multi-layer capacity, interaction capacity, topology capacity).

6 Replication Tasks: Anchoring in Other Papers

Alongside our own experiments, we plan to **replicate or partially reproduce** key results from the following works. These replications ground our claims in the broader literature and show where our geometric perspective fits.

1. **Toy Models of Superposition (Elhage et al., 2022)** [1]. Replicate the toy ReLU networks with sparse features showing superposition, phase transitions, and geometric structures; borrow their setup for our T1 experiments.
2. **Superposition Yields Robust Neural Scaling (Liu et al.)** [2]. Replicate the toy-model scaling curves: strong vs. weak superposition regimes, loss vs. width, and feature-frequency distributions. Confirm that our KD setups operate in a strong-superposition regime by measuring feature counts vs. width.

3. **Geometry-Aware Representational Alignment (Bhattarai et al.)** [5]. Reproduce their observation that MSE/CKA can fail to capture feature structure and that Procrustes/Gram losses help; integrate their losses into our KD experiments to see how geometry-aware alignment interacts with capacity limits. Their public code [15] provides Procrustes/Gram KD implementations for BERT/OPT tasks.
4. **Interaction-Based KD Interpretation (Wang et al.)** [6]. Reproduce the basic findings that KD sparsifies complex interactions and that higher interaction sparsity correlates with better KD performance; compare interaction-sparsity metrics with our feature-capacity metrics in at least one shared setting.
5. **KD and LLM surveys** [8, 9]. Use survey categorizations to ensure our KD setups reflect realistic pipelines and open challenges (e.g., emergent abilities, domain-specific KD, the capacity-gap problem).

7 Justification: Why This Project Makes Sense Now

7.1 Field Needs and Open Problems

Surveys and reviews identify several open problems:

- Lack of a unified theory of KD limits across architectures and distillation strategies [8, 9].
- Difficulty preserving emergent reasoning and long-tail linguistic diversity during LLM KD [9].
- The need for mechanistic understanding of what gets transferred or lost in KD [13].

Our project responds by:

- Providing a **capacity-based, geometric lens** on when and why KD floors appear, grounded in superposition and compressed sensing [1, 2, 3].
- Integrating mechanistic interpretability tools (SAEs, feature probes) into KD analysis [13, 14].
- Connecting to newer geometric and interaction-based KD views, aiming for a more unified picture [5, 6, 7].

7.2 Why Width and Superposition?

Recent work suggests that LLMs operate in strong-superposition regimes, where representations pack many features into fewer dimensions, making width a key driver of scaling laws and loss [2]. Compressed sensing and sparse-representation theory give principled formulas for capacity, which can be adapted to KD [3]. Studying **width-driven geometric limits in KD** is therefore both natural and timely.

8 Process and Workflow

8.1 Code and Reproducibility

We will build a modular codebase with distinct components, use versioned configs and seeds for all experiments, and plan to release the code once the pipeline is stable. The planned repository structure:

Listing 4: Planned repository layout for `geometric-kd`.

```
geometric-kd/
  experiments_matrix.md
  toy/
    data.py
    models.py
    train_kd.py
  lm/
    bert_opt_kd.py
    losses_geom.py
    sae_features.py
  vision_gnn/
    kd_vision.py
    kd_gnn.py
  analysis/
    capacity_estimation.py
    plots.ipynb
```

Component responsibilities: `toy/` implements the T1 data and models based on *Toy Models of Superposition* [1]; `lm/` contains KD training scripts, geometry-aware losses, and SAE feature-measurement tools for the LM tiers; `vision_gnn/` covers T4; and `analysis/` holds capacity estimation, loss-floor prediction, and evaluation scripts.

8.2 Iterative Development

1. **Phase 1 — Toy + LM audit (Months 1–2).** Reimplement the toy superposition model and LM KD with a clean pipeline; run sensitivity analyses; log deviations from our earlier claims [16].
2. **Phase 2 — New architectures and KD objectives (Months 3–4).** Add vision/GNN KD experiments; integrate geometry-aware and interaction-aware losses [5, 6, 7].
3. **Phase 3 — Synthesis and writing (Months 5+).** Refine the theory sections informed by experimental outcomes; decide which claims remain as theorems, which are downgraded to models/conjectures, and which are rejected.

8.3 Iteration and Documentation

For each experiment, we write a short note: “*What was tested, what did we expect from theory, and what actually happened?*” We use these notes to refine Model M1 and Hypothesis H1:

- In some settings, we may add terms or modify the capacity function $g(\alpha)$.
- In others, we may identify that interaction-based or geometry-only explanations work better.

9 Immediate Next Steps

1. **Revisit our earlier write-up** and annotate which sections are kept, revised, or removed [16].
2. **Create the repository structure** shown in Section 8.
3. **Define `experiments_matrix.md`** with concrete experimental configurations (models, widths, datasets, KD objectives, hyperparameters, seeds), starting from Table 1.

4. **Start with toy/**: implement the toy superposition model and feature-measurement pipeline, anchored to the setups of Elhage et al. and Liu et al. [1, 2], logging loss vs. width and simple feature-capacity metrics.
5. **Clone the feature-distillation repository** [15]: adapt its Procrustes/Gram KD code into the `lm/` module and use its training configs as a baseline [5].

From here, we iteratively refine both theory and experiments, always checking against current KD and mechanistic interpretability papers to ensure our claims are justified by the literature and the data. We can extend each section as experiments begin running, and flesh out `experiments_matrix.md` with precise hyperparameters once exact models and datasets are fixed.

References

- [1] N. Elhage, T. Hume, C. Olsson, et al. Toy models of superposition. *Transformer Circuits Thread*, Anthropic, 2022.
- [2] Y. Liu, et al. Superposition yields robust neural scaling. arXiv preprint, 2025.
- [3] D. L. Donoho. Compressed sensing. *IEEE Transactions on Information Theory*, 52(4):1289–1306, 2006.
- [4] K. Park, Y. J. Choe, and V. Veitch. The linear representation hypothesis and the geometry of large language models. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.
- [5] M. Bhattarai, et al. Geometry-aware representational alignment for feature distillation (Procrustes- and Gram-based feature KD losses). arXiv preprint.
- [6] Wang, et al. A unified interaction-based interpretation of knowledge distillation: distillation sparsifies complex interactions. arXiv preprint.
- [7] C. Yang, Q. Wu, and J. Yan. Geometric knowledge distillation: Topology compression for graph neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [8] J. Gou, B. Yu, S. J. Maybank, and D. Tao. Knowledge distillation: A survey. *International Journal of Computer Vision*, 129:1789–1819, 2021.
- [9] X. Xu, M. Li, C. Tao, et al. A survey on knowledge distillation of large language models. arXiv preprint arXiv:2402.13116, 2024.
- [10] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. arXiv preprint arXiv:1503.02531, 2015.
- [11] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2019.
- [12] S. Biderman, H. Schoelkopf, Q. Anthony, et al. Pythia: A suite for analyzing large language models across training and scaling. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.
- [13] L. Bereska and E. Gavves. Mechanistic interpretability for AI safety: A review. *Transactions on Machine Learning Research*, 2024.

- [14] T. Bricken, A. Templeton, J. Batson, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, Anthropic, 2023.
- [15] Feature-distillation repository (Procrustes/Gram KD implementations for BERT/OPT tasks). GitHub, <https://github.com/x-labs-xyz/feature-distillation>.
- [16] Geometric limits of knowledge distillation. Earlier internal notes and preliminary results from this ongoing project.